Introduction aux Bases de Données Relationnelles et Requêtes SQL de Base

Thème principal : Modèle Relationnel, Clés Primaires/Étrangères et Jointures SQL ANSI

Synthèse exécutive : Ce cours pose les fondations indispensables de la ressource R1.05 (Introduction aux Bases de Données). L'objectif pédagogique est de comprendre la transition fondamentale des tableurs plats (Excel) vers les Systèmes de Gestion de Bases de Données Relationnelles (SGBD). Le cours détaille comment structurer l'information sans redondance grâce aux clés primaires et clés étrangères tout en garantissant l'intégrité référentielle, puis enseigne l'écriture de requêtes SQL d'extraction fiables et normalisées (norme ISO/ANSI) en utilisant les jointures explicites INNER JOIN.

1. Le Pourquoi : Des Tableurs aux Bases de Données Relationnelles

Dans une application informatique réelle, l'utilisation de fichiers plats ou de feuilles de calcul Excel montre très rapidement ses limites. Si un client passe 50 commandes et que son adresse est répétée sur chaque ligne de commande, le moindre changement d'adresse nécessite 50 modifications manuelles. Si une seule ligne est oubliée, la base souffre d'une anomalie de mise à jour et devient incohérente. Le modèle relationnel résout ce problème à la racine : les entités distinctes ('Clients' et 'Commandes') sont séparées dans des tables spécifiques. La relation entre un client et ses commandes s'effectue par des pointeurs logiques, évitant ainsi toute duplication inutile de données.

- Tableur plat vs Base relationnelle : Problématique de la redondance et des anomalies de mise à jour.
- SGBD (Système de Gestion de Bases de Données) : Logiciel garantissant la cohérence, la sécurité, l'accès concurrentiel et la persistance des données.
- Modèle Relationnel : Découpage des données en entités (tables) reliées logiquement pour éliminer les doublons.

Cas concret enseignant : *Exemple du système bancaire : Si une banque enregistrait le nom, prénom et adresse du titulaire sur chaque ligne de virement ou de paiement par carte, un simple déménagement exigerait la mise à jour de milliers de lignes d'historique bancaire, générant un risque majeur d'erreur et une saturation inutile du stockage.*

2. Clés Primaires, Clés Étrangères et Intégrité Référentielle

Pour garantir qu'aucun enregistrement ne soit confondu avec un autre, chaque table doit posséder une Clé Primaire. Cet attribut (ou ensemble d'attributs) garantit l'unicité absolue de la ligne. Lorsqu'on souhaite lier deux tables (par exemple lier un Étudiant à son Groupe de TD), on n'insère pas toutes les informations du groupe dans la table Étudiant : on y stocke simplement l'identifiant du groupe sous forme de Clé Étrangère. Le SGBD applique alors automatiquement une contrainte d'intégrité référentielle : il est impossible d'attribuer un numéro de groupe à un étudiant si ce groupe n'existe pas dans la table des groupes.

- Clé Primaire (Primary Key - PK) : Identifiant unique de chaque ligne (n-uplet), obligatoire et non nul (NOT NULL).
- Clé Étrangère (Foreign Key - FK) : Attribut stocké dans une table qui fait référence à la clé primaire d'une autre table.
- Contrainte d'Intégrité Référentielle : Règle stricte gérée par le SGBD qui interdit d'insérer une clé étrangère pointant vers un identifiant inexistant.

Cas concret enseignant : *Exemple du Numéro de Sécurité Sociale (NIR) / INSEE à Bayonne : Même si deux étudiants portent exactement le même nom et le même prénom au Pays Basque, leur clé primaire unique (NIR ou id_etudiant) empêche toute confusion au sein du système d'information de l'IUT.*

3. Extraction de Données en SQL ANSI : SELECT, WHERE et INNER JOIN

Le langage SQL (Structured Query Language) permet d'interroger la base de données. Pour croiser des informations entre deux tables, la norme moderne ISO/ANSI exige l'utilisation de la clause explicite INNER JOIN

associée à la condition de liaison ON. La syntaxe standard s'écrit : SELECT colonnes FROM table1 INNER JOIN table2 ON table1.fk = table2.pk WHERE conditions. L'utilisation des alias de tables (ex: 'Etudiant e') est fortement recommandée pour rendre la requête lisible et éviter les ambiguïtés de nommage de colonnes.

- Langage Déclaratif : SQL exprime CE QUE l'on cherche et non COMMENT l'extraire.
- Projection (SELECT) : Sélection explicite des colonnes à retourner.
- Filtrage (WHERE) : Application de conditions logiques d'exclusion ou d'inclusion sur les lignes.
- Jointure Explicite (INNER JOIN ... ON) : Combinaison de tables basée sur l'égalité entre Clé Étrangère et Clé Primaire.

Cas concret enseignant : *Mise en garde stricte du professeur pour les examens : 'N'utilisez jamais la syntaxe obsolète du produit cartésien avec une virgule dans le FROM (ex: FROM Etudiant, Groupe WHERE Etudiant.id_g = Groupe.id). Utilisez TOUJOURS la syntaxe explicite INNER JOIN ... ON. Le produit cartésien implicite fait perdre des points en partiel et est banni en entreprise.'*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi ne doit-on pas utiliser SELECT * dans un code applicatif ou lors d'un examen ?
Réponse du professeur : L'utilisation de 'SELECT *' est une mauvaise pratique majeure pour trois raisons : 1) Performance : elle force la base de données et le réseau à transférer des colonnes inutiles. 2) Fragilité du code : si la structure de la table change (ajout ou suppression d'une colonne), l'application peut planter si elle s'attend à un ordre fixe. 3) Clarté : en examen, le correcteur veut vérifier que l'étudiant maîtrise la projection explicite des champs demandés.

Question étudiant : Que se passe-t-il si j'essaie de supprimer un groupe dans lequel il y a encore des étudiants inscrits ?
Réponse du professeur : Grâce à la contrainte d'intégrité référentielle, le SGBD intercepte l'opération et bloque la suppression en renvoyant une erreur d'intégrité. Cela empêche d'avoir des 'étudiants orphelins' pointant vers un groupe qui n'existe plus. Pour pouvoir supprimer le groupe, il faut soit réaffecter/supprimer les étudiants au préalable, soit configurer des règles spécifiques de suppression en cascade (ON DELETE CASCADE).

■ Exercice & Démonstration d'Application

Énoncé : On considère le schéma relationnel suivant : - Etudiant (id_etudiant, nom, prenom, #id_groupe) - Groupe (id_groupe, nom_groupe, promo) Écrire la requête SQL conforme à la norme ISO/ANSI permettant d'extraire le nom et le prénom des étudiants appartenant à la promotion 'BUT1-INFO' et inscrits dans le groupe nommé 'A1'.

Résolution pas à pas :
Étape 1 : Identifier les tables nécessaires et définir leurs alias -> Etudiant (e) et Groupe (g). Étape 2 : Définir la jointure explicite sur la correspondance clé étrangère / clé primaire -> INNER JOIN Groupe g ON e.id_groupe = g.id_groupe. Étape 3 : Appliquer les filtres dans la clause WHERE -> g.promo = 'BUT1-INFO' AND g.nom_groupe = 'A1'. Étape 4 : Sélectionner uniquement les attributs demandés (projection) -> SELECT e.nom, e.prenom. Requête SQL rédigée : SELECT e.nom, e.prenom FROM Etudiant e INNER JOIN Groupe g ON e.id_groupe = g.id_groupe WHERE g.promo = 'BUT1-INFO' AND g.nom_groupe = 'A1';

Résultat : La requête retourne la liste précise des noms et prénoms des étudiants du groupe A1 en BUT1-INFO. Elle est optimale, lisible, respecte la norme ANSI et évite tout produit cartésien.

■ Auto-Évaluation Flash d'Examen

Q : Quelles sont les deux propriétés fondamentales d'une Clé Primaire (Primary Key) ?

R : Une clé primaire doit être strictement unique pour chaque enregistrement et ne peut jamais contenir de valeur nulle (NOT NULL).

Q : À quoi sert la clause INNER JOIN et sur quoi doit porter sa condition ON ?

R : INNER JOIN sert à combiner logiquement les lignes de deux tables. Sa condition ON doit exprimer l'égalité entre la Clé Étrangère de l'une et la Clé Primaire de l'autre.

Q : Pourquoi la syntaxe 'FROM TableA, TableB WHERE TableA.id = TableB.id_a' fait-elle perdre des points en examen ?

R : Parce qu'il s'agit d'une syntaxe implicite basée sur un produit cartésien non explicite, obsolète depuis la norme SQL-92. La norme exigée est l'utilisation explicite de 'INNER JOIN ... ON'.